

JOBSHEET II

Mata Kuliah: Praktikum Pemrograman Berbasis Objek

Pemrograman Berorientasi Objek

Dosen Pengampu:

Dian Hanifudin Subhi, S.Kom., M.Kom.



Disusun oleh:

Ahmad Rafid Riqkullah

254107020078

T1 – 2B

D-IV Teknik Informatika

Tanggal: 4 September 2026

POLITEKNIK NEGERI MALANG

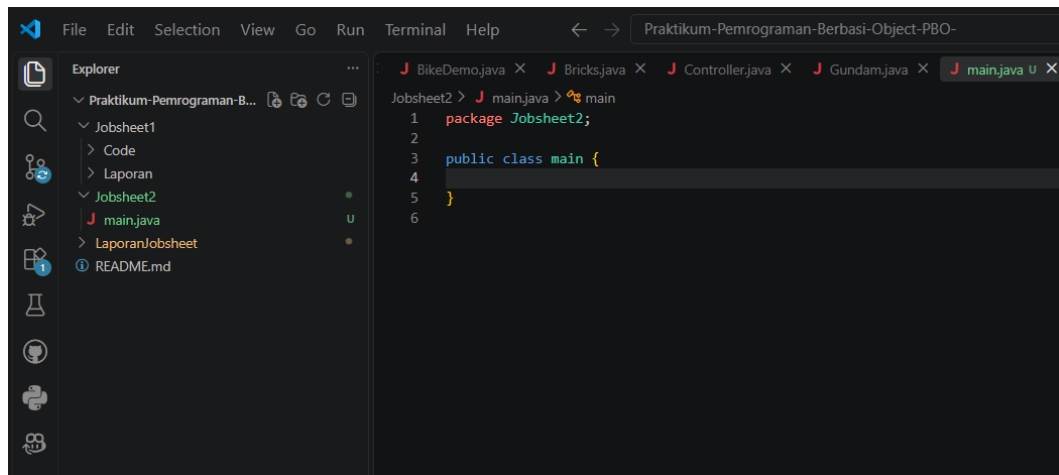
Jl. Soekarno Hatta No.9, Jatimulyo, Kec. Lowokwaru, Kota Malang, Jawa Timur

65141

TAHUN 2026-2027

C. Langkah Kerja

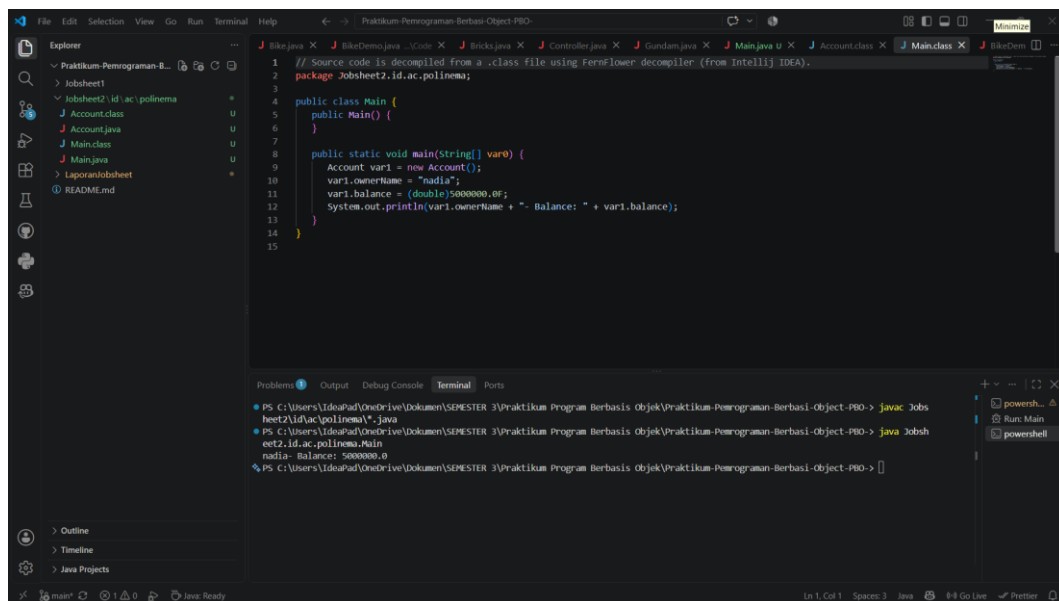
Langkah 1: Proyek Bank-mini



Gambar 1 Struktur Proyek

Penjelasan: Langkah ini bertujuan untuk menyiapkan struktur awal proyek (package) dan membuat class Main yang berisi method main. Method main ini berfungsi sebagai titik awal eksekusi program.

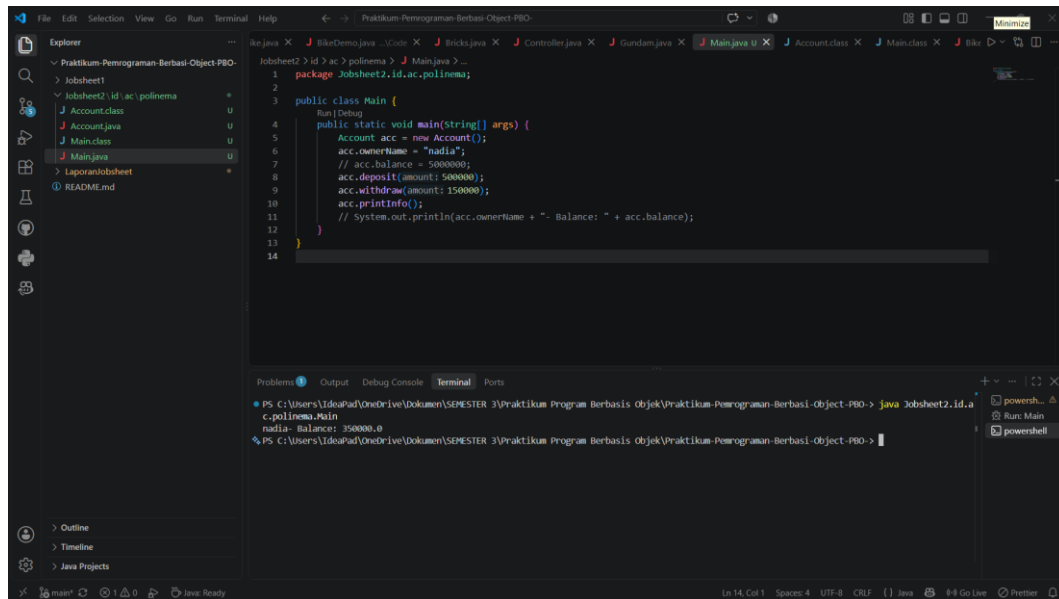
Langkah 2: Class Account



Gambar 2 Langkah 2: Class Account

Penjelasan: Pada langkah ini, kita membuat class Account yang bertindak sebagai *blueprint* (cetakan) untuk objek rekening bank. Class ini dideklarasikan dengan dua atribut, yaitu ownerName (bertipe String untuk nama pemilik) dan balance (bertipe double untuk jumlah saldo).

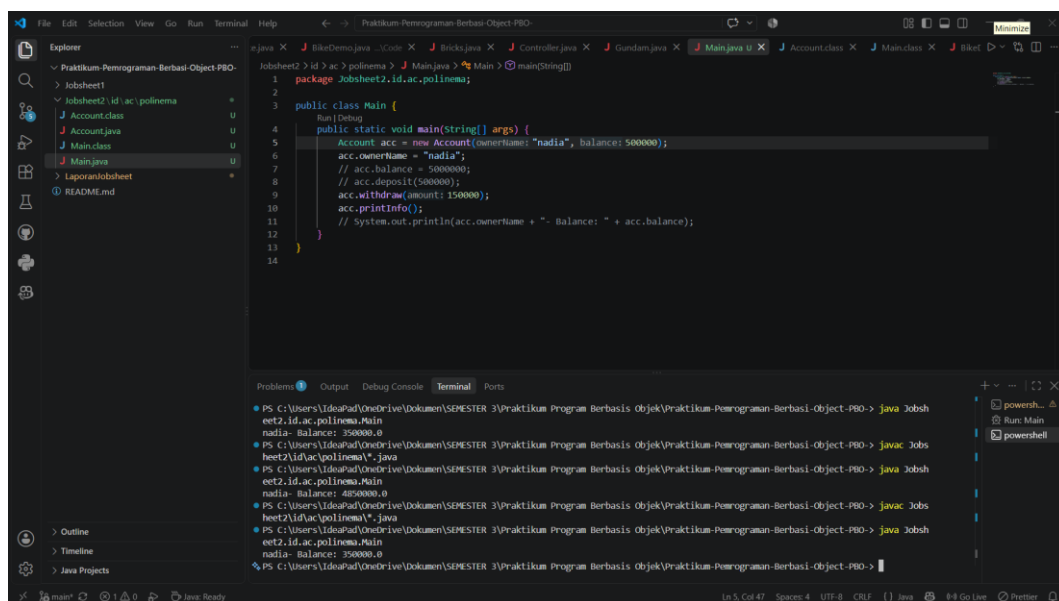
Langkah 3: Menambahkan Method



Gambar 3 Langkah 3: Menambahkan Method

Penjelasan: Class Account ditambahkan perilaku (*behavior*) melalui method. Terdapat method deposit() untuk menambah saldo, withdraw() untuk mengurangi saldo, dan printInfo() untuk menampilkan informasi nama pemilik dan saldo saat ini ke layar.

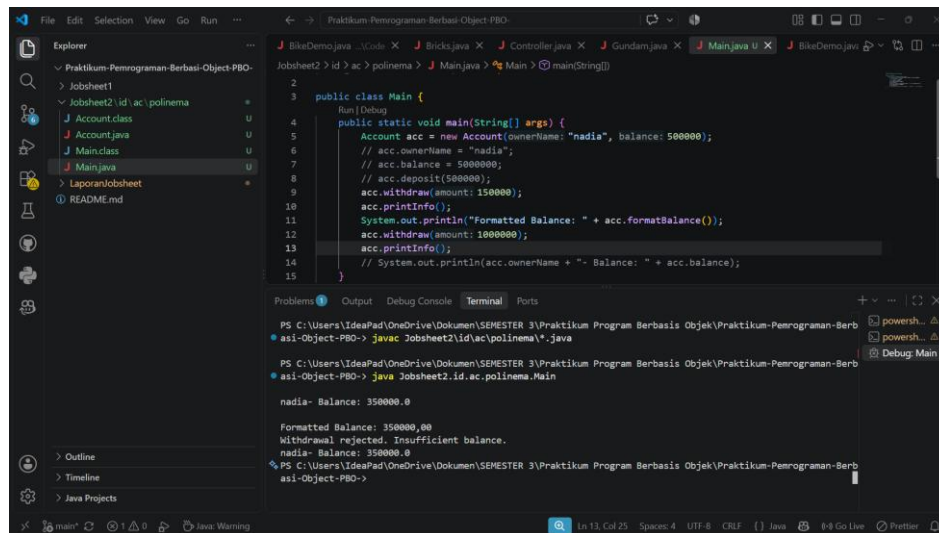
Langkah 4: Konstruktor



Gambar 4 Langkah 4: Konstruktor

Penjelasan: Sebuah konstruktor berparameter ditambahkan ke class Account. Konstruktor ini bertugas untuk memberikan nilai awal (*inisialisasi*) pada atribut ownerName dan balance tepat ketika objek pertama kali dibuat. Keyword this digunakan untuk membedakan antara atribut milik class dengan parameter dari konstruktor.

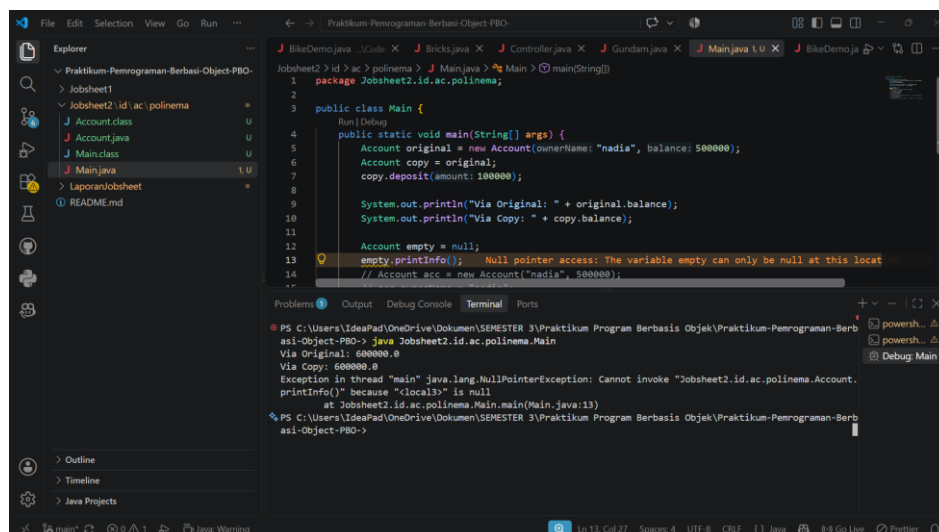
Langkah 5: Method dengan Logika dan Nilai Kembali



Gambar 5 Langkah 5: Method dengan Logika dan Nilai Kembali

Penjelasan: Langkah ini menambahkan method yang mengembalikan nilai (*return value*), yaitu formatBalance() untuk memformat angka saldo (menjadi String) dan isOverDrawn() untuk mengecek (boolean) apakah saldo negatif. Logika pada method withdraw() juga dimodifikasi dengan pengkondisian if (isOverDrawn()) untuk menolak penarikan yang menyebabkan saldo kurang dari nol.

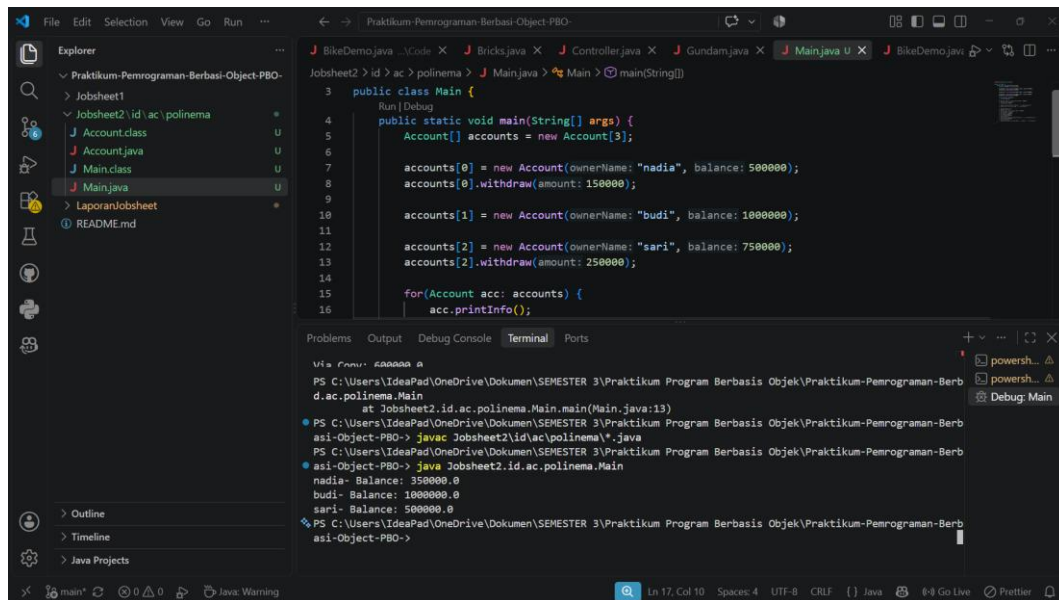
Langkah 6: Referensi, Aliasing, dan null



Gambar 6 Langkah 6: Referensi, Aliasing, dan null

Penjelasan: Langkah ini mendemonstrasikan konsep *aliasing*, di mana variabel copy menunjuk ke referensi objek yang sama dengan original, sehingga perubahan pada copy akan berdampak pada original. Selain itu, variabel empty diisi dengan null (tidak merujuk ke objek mana pun), sehingga saat dipanggil method-nya, program menghasilkan error `NullPointerException`.

Langkah 7: Array of Objects, Banyak Objek dari Satu Kelas

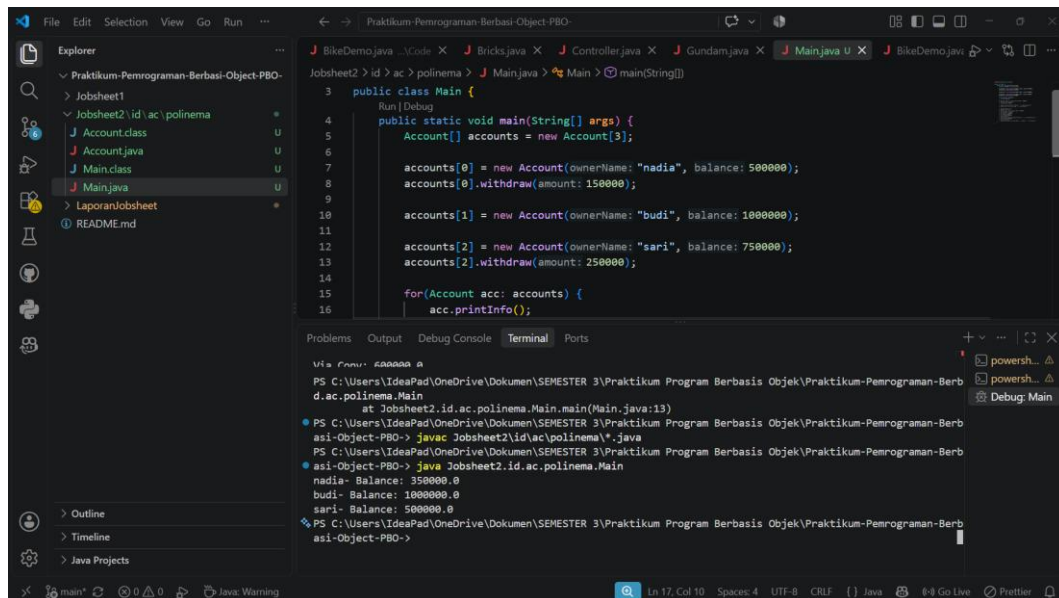


Gambar 7 Langkah 7: Array of Objects, Banyak Objek dari Satu Kelas

Penjelasan: Langkah ini menunjukkan cara mengelola banyak objek menggunakan *Array of Objects*. Array `accounts` dibuat untuk menyimpan 3 objek `Account` yang berbeda (Nadia, Budi, dan Sari). Pemanggilan method seperti `printInfo()` dapat dilakukan secara efisien menggunakan perulangan *for-each*.

D. Tugas dan Deliverable

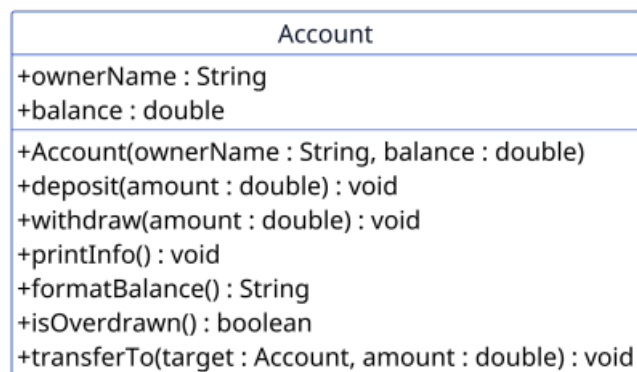
Scrennsnot output program Langkah 7.



Gambar 8 Screenshoot Langkah 7

Tugas mandiri: 1.

1. Bank Mini perlu memindahkan saldo antar rekening. Tambahkan method `transferTo` ke `Account` sesuai diagram kelas UML berikut. Tidak ada contoh kode untuk langkah ini, rancang sendiri isinya berdasarkan method yang sudah tersedia (`deposit()`, `withdraw()`) pada diagram:



Gambar 9 Diagram kelas UML untuk Account dengan tambahan `transferTo`

```

1  package Jobsheet2.id.ac.polinema;
2
3  public class Account {
4      public String ownerName;
5      public double balance;
6
7      public Account(String ownerName, double balance) {
8          this.ownerName = ownerName;
9          this.balance = balance;
10     }
11
12     public void deposit(double amount) {
13         balance = balance + amount;
14     }
15
16     public void withdraw(double amount) {
17         balance = balance - amount;
18         if (isOverDrawn()) {
19             balance = balance + amount;
20             System.out.println("Withdrawal rejected. Insufficient balance.");
21         }
22     }
23
24     public void printInfo() {
25         System.out.println(ownerName + "- Balance: " + balance);
26     }
27
28     public String formatBalance() {
29         return String.format("%.2f", balance);
30     }
31
32     public boolean isOverDrawn() {
33         return balance < 0;
34     }
35
36     public void transferTo(Account target, double amount) {
37         if(this.balance >= amount) {
38             this.withdraw(amount);
39             target.deposit(amount);
40         } else {
41             this.withdraw(amount);
42         }
43     }
44 }
45

```

Gambar 10 Code Account.java

Penjelasan: Method transferTo ditambahkan dengan dua parameter: target (objek rekening tujuan) dan amount (jumlah transfer). Logikanya adalah mengecek apakah saldo cukup. Jika cukup, maka method withdraw() dijalankan pada akun pengirim, dan method deposit() dijalankan pada akun target. Jika tidak cukup, maka hanya memanggil withdraw() agar tercetak pesan penolakan penarikan.

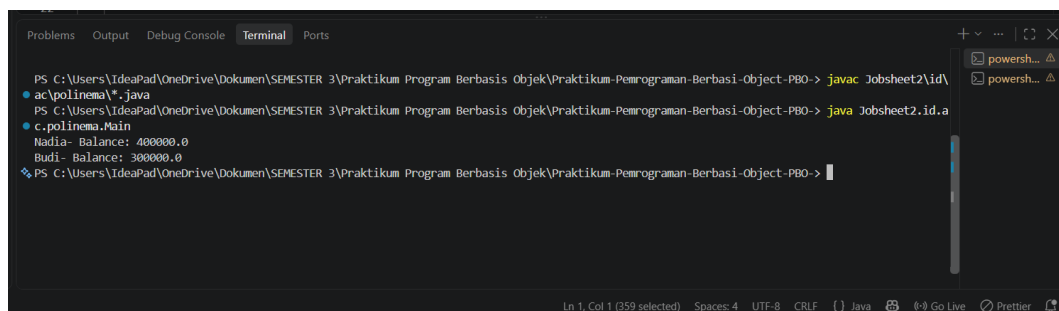
Buktikan dengan membuat dua objek Account di Main, memanggil transferTo dari salah satunya ke yang lain, lalu mencetak keduanya:



```
1 package Jobsheet2.id.ac.polinema;
2
3 public class Main {
4     public static void main(String[] args) {
5         // untuk Tugas mandiri
6         Account from = new Account("Nadia", 500000);
7         Account to = new Account("Budi", 200000);
8
9         from.transferTo(to, 100000);
10
11         from.printInfo();
12         to.printInfo();
13     }
14 }
```

Gambar 11 Code Main.java

Penjelasan: Pada class Main, dilakukan pengujian dengan menginstansiasi dua objek Account, yaitu "Nadia" dengan saldo 500.000 dan "Budi" dengan saldo 200.000. Kemudian, method `from.transferTo(to, 100000)` dipanggil untuk memindahkan saldo sebesar 100.000 dari rekening Nadia ke rekening Budi.



```
PS C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 3\Praktikum Program Berbasis Objek\Praktikum-Pemrograman-Berbasi-Object-PBO-> javac Jobsheet2\id\ac\polinema\*.java
PS C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 3\Praktikum Program Berbasis Objek\Praktikum-Pemrograman-Berbasi-Object-PBO-> java Jobsheet2.id.ac.polinema.Main
Nadia- Balance: 400000.0
Budi- Balance: 300000.0
PS C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 3\Praktikum Program Berbasis Objek\Praktikum-Pemrograman-Berbasi-Object-PBO-> |
```

Gambar 12 Hasil Percobaan

Penjelasan: Output program membuktikan bahwa transfer berhasil dilakukan. Saldo milik Nadia (sebagai pengirim) berkurang 100.000 menjadi 400.000, sedangkan saldo milik Budi (sebagai penerima) bertambah 100.000 menjadi 300.000.

2. Jawab secara singkat (2-3 kalimat untuk masing-masing pertanyaan):
 - a. apa perbedaan antara objek dengan referensi ke objek?

Jawab: Objek adalah entitas nyata yang ada di dalam ruang memori (heap) yang memiliki atribut dan perilaku. Sedangkan referensi ke objek adalah sebuah variabel yang menyimpan alamat memori tempat objek tersebut berada, yang kita gunakan untuk mengakses atau memodifikasi objek tersebut.

b. kapan tepatnya konstruktor sebuah kelas dijalankan?

Jawab: Konstruktor dijalankan tepat satu kali secara otomatis pada saat pembuatan (instansiasi) sebuah objek baru di memori, yaitu ketika kita menggunakan *keyword* new (contoh: new Account(...)).

Hasil Dan Pembahasan

Pada Jobsheet 2 ini, program sistem "Bank-mini" berhasil dibuat menggunakan konsep dasar PBO. Kita membuat sebuah cetakan (class) bernama Account yang menyimpan data nama pemilik dan jumlah saldo.

Class ini punya beberapa fungsi (method) untuk melakukan aksi, seperti menabung (deposit()), menarik uang (withdraw()), dan melihat info saldo (printInfo()). Kita juga menggunakan *konstruktor* untuk langsung mengisi nama dan saldo awal saat akun baru dibuat.

Pada file Main, kita mencoba membuat banyak akun sekaligus menggunakan *Array*. Selain itu, lewat tugas membuat method transferTo, kita belajar bahwa antar objek bisa saling berinteraksi. Saat akun Nadia mentransfer uang ke akun Budi, program sebenarnya mengirimkan "alamat" objek Budi ke dalam fungsi, sehingga saldo Budi bisa ikut bertambah.

Kesimpulan

Kesimpulannya, Pemrograman Berorientasi Objek (PBO) sangat membantu kita merancang kode yang cara kerjanya mirip dengan dunia nyata. Class bertugas sebagai bungkus yang menyatukan data (atribut) dan aksi (method) di satu tempat.

Selain itu, hal terpenting yang dipelajari adalah variabel objek di Java sebenarnya hanya menyimpan "alamat referensi" di memori. Karena konsep referensi inilah kita bisa memindahkan data antar objek (seperti proses transfer uang) atau memunculkan error seperti *NullPointerException* jika sebuah variabel kosong dan tidak menunjuk ke alamat objek mana pun.

Lampiran

[Link Github Ahmad Rafid Riqkullah](https://github.com/Pidddd/Praktikum-Pemrograman-Berbasi-Object-PBO-/tree/main/Jobsheet2/id/ac/polinema) atau <https://github.com/Pidddd/Praktikum-Pemrograman-Berbasi-Object-PBO-/tree/main/Jobsheet2/id/ac/polinema>